

Anwendungsorientierte Softwareentwicklung

Laborblatt 11: Levels, Leben, Vielfalt und Kontrolle

Lernziele

- Ein Softwareprojekt unter realistischen Bedingungen (sprich: kurz vor der Deadline) zu einem spielbaren, beeindruckenden Abschluss bringen
- Die Vorteile von `Enum`-Konstanten für Klarheit, Erweiterbarkeit und Lesbarkeit im Code spontan erkennen und praktisch nutzen
- Den neuen Gyro- und Beschleunigungssensor-Controller mit einem „Wow-Faktor“ ins Spiel bringen – und dem Kunden (oder dir) schmackhaft präsentieren

Vorbereitung

Wie üblich arbeiten wir mit zwei Terminals:

- **links**: Quellcode editieren
- **rechts**: Spiel testen

Klonen des Repositories:

```
git clone https://gitlab.uni-ulm.de/BENUTZERNAME/asp-ulm
cd asp-ulm/python
python breakout.py
```

Macht euch wie immer **kurz damit vertraut**, welche Features euer Spiel aktuell unterstützt.

Falls es **unerwartet zu Abstürzen kommt** – und ihr das **Glück habt**, sie jetzt zu entdecken – dann behebt sie am besten jetzt gleich. **Gerne mit unserer Hilfe.**

Schritt 1: Bomben sind nicht zur Zierde da

Wenn das Paddle von einer Bombe getroffen wird, soll es für eine gewisse Zeit **außer Gefecht** sein. In diesem Schritt überprüfen wir zunächst nur, ob eine Bombe das Paddle trifft. Wenn ja, soll die Bombe zünden.

Dazu könnt ihr **ähnlich vorgehen** wie bei der Überprüfung, ob ein Ball das Paddle trifft.

Falls ihr es richtig gut machen wollt: Werft einen Blick in `ball.py`. Dort wird in der `move`-Methode überprüft, ob der **Mittelpunkt** des Balls die **obere Kante** des Paddles durchquert.

Wenn ihr analog vorgeht, könnt ihr in `bomb.py` die `move`-Methode entsprechend anpassen. Der Code wird hier sogar **etwas einfacher**, weil unsere Bomben immer **senkrecht nach unten** fliegen.

Wird das Paddle getroffen, dann setzt ihr einfach das Attribut `self.hit = True`. Die `draw`-Methode kümmert sich – **ohne Änderungen** – um den Rest.

Eure `move`-Methode in `bomb.py` könnte dann z. B. so aussehen:

```
def move(self):
    if self.y < 0 or self.y > Game.height:
        self.alive = False
        return
    new_y = self.y + self.vy
    if self.y < Paddle.y-10 and new_y >= Paddle.y-10:
        if self.x >= Paddle.x and self.x <= Paddle.x + Paddle.w:
            self.hit = True
    self.y = new_y
```

Aufgabe: Testet, ob nach eurer Änderung die Bomben explodieren, wenn sie das Paddle treffen.

Schritt 2: Bomben sind schlecht für's Paddle

Wenn das Paddle getroffen wurde, soll das **Auswirkungen** haben: Für eine gewisse Zeit ist es **außer Gefecht** – und insbesondere **nicht sichtbar**. Hier folgt eine kleine **Vorgabe**, wie man das Feature **zumindest teilweise** umsetzen kann. Den Rest werdet ihr im Laufe der Aufgabe selbst erweitern.



In `paddle.py`

Das Paddle erhält ein neues statisches Attribut `damage`, das mit `0` initialisiert wird. Es gibt an, für **wie viele Frames** das Paddle nicht einsetzbar ist. Die Klasse `Paddle` beginnt also z. B. so:

```
class Paddle:
    c = (0, 0, 255)
    w, h = 100, 30
    x, y = None, None
    damage = 0
```



Neue Methode `hit`

Die Methode `hit()` „zerstört“ das Paddle – **aber nur, wenn es noch intakt ist**. Falls es bereits beschädigt ist, passiert nichts.

```
def hit():
    if Paddle.damage == 0:
        Paddle.damage = 90
        return True
    return False
```

Hinweis: Die Methode gibt `True` zurück, wenn das Paddle gerade beschädigt wurde. Sonst `False` (z. B. wenn es eh schon kaputt ist).



Anpassung der `draw`-**Methode**

Wenn das Paddle beschädigt ist, wird es nicht gezeichnet – aber der Schaden verringert:

```
def draw():
    if Paddle.damage > 0:
        Paddle.damage -= 1
        return
    pygame.draw.rect(Game.screen, Paddle.c,
                     (Paddle.x, Paddle.y, Paddle.w, Paddle.h))
```

Aufgabe

1. Passt den Code in `paddle.py` wie oben beschrieben an. Versucht dabei wie immer zu verstehen, was genau passiert. Jetzt soll natürlich `Paddle.hit()` aufgerufen werden, wenn ein intaktes Paddle von einer Bombe getroffen wird.

Ändert dazu in `bomb.py` die Zeile `self.hit = True` in `self.hit = Paddle.hit()`. Damit wird `self.hit` nur dann auf `True` gesetzt, wenn das Paddle durch den Bombentreffer tatsächlich beschädigt wurde.

2. Das Paddle ist jetzt zwar eine Zeitlang unsichtbar, wenn es getroffen wurde – aber es ist trotzdem noch da. Das heißt: Selbst im beschädigten Zustand kann es noch Bälle reflektieren. Deshalb jetzt zwei Teilaufgaben:

(a) (schwierig) Beobachtet das Verhalten

Versucht, das Problem zunächst **bewusst zu beobachten**, bevor ihr es behebt. **Schafft ihr es, mit einem unsichtbaren Paddle einen Ball abprallen zu lassen?** Das ist gar nicht so einfach – aber genau diese Art von Verhalten soll ein gutes Testteam entdecken

(b) (leicht) Behebt das Problem

Passt in `ball.py` die Stelle an, an der überprüft wird, ob der Ball mit dem Paddle kollidiert. Fügt dort eine zusätzliche Bedingung ein, sodass der Ball **nur dann reflektiert** wird, wenn das Paddle **intakt ist**.

3. Behebt jetzt noch das Problem, dass man aktuell noch mit einem zerstörten Paddle Raketen abfeuern kann.

Schritt 3: Es lebe die Vielfalt (Teil 1 von 3)

Aktuell sind unsere Bricks noch recht **eintönig**: Sie sind entweder **da oder weg** – mehr nicht. Das soll sich jetzt ändern. Es soll **verschiedene Brick-Arten** geben, zum Beispiel:

- Bricks, die man **zweimal treffen** muss (erst dann verschwinden sie)
- Bricks, die beim Zerstören **einen neuen Ball** ins Spiel bringen
- Bricks, die dem Paddle **einen temporären Schutzschild** geben – danach ist man für eine Weile immun gegen Bomben
- Und vielleicht noch mehr...

Idee: Zahlen für Brick-Typen

Die Art eines Bricks kann man zunächst mit Zahlen codieren:

- 0 → kein Brick
- 1 → normaler Brick (was wir bisher haben)
- 2 → Brick, der zweimal getroffen werden muss
- 3 → Brick, der einen neuen Ball erzeugt
- 4 → Brick, der einen Schutzschild aktiviert

Das hat den Vorteil, dass man wie bisher mit einem `numpy`-Array oder Vergleichsoperatoren arbeiten kann.

Aber: Zahlen allein sind nicht besonders sprechend – man vergisst schnell, wofür 3 oder 4 nochmal standen.

Abhilfe: `enum`-Typen

Hier kommt ein Mini-Crashkurs – ganz im Stil von: „**Learning by using and playing around.**“

Legt eine neue Datei namens `bricktype.py` mit folgendem Inhalt an:

```
1 from enum import IntEnum
2
3 class BrickType(IntEnum):
4     NONE = 0
5     NORMAL = 1
6     BALL = 2
7     SHIELD = 3
8     TWICE = 4
9
10     def label(self):
11         return f"Dummer Label für {self}"
12
13 def main():
14     print(max(BrickType))
15     print(BrickType.BALL)
16     print(BrickType.BALL.label())
17     foo = BrickType(4)
18     print(foo)
19     print(foo.label())
20
21
22 if __name__ == "__main__":
23     main()
```

Erläuterungen

- In **Zeile 3** wird `BrickType` als Erweiterung von `IntEnum` deklariert. Wir gehen an dieser Stelle **nicht im Detail darauf** ein, was das bedeutet – **außer** in einer späteren **Teilaufgabe**, bei der ihr das `(IntEnum)` einmal entfernt, um zu sehen, **was dann plötzlich nicht mehr geht**.
- In **Zeile 4–8** werden die verschiedenen **Brick-Arten** als Konstanten mit **sprechenden Namen** festgelegt – deutlich lesbarer als reine Zahlen.
- In **Zeile 10–11** wird eine Methode `label` definiert, die erstmal nur einen **Platzhalter-Text** liefert – aber gleich zeigt, **was mit Enums möglich ist**.

Aufgabe

1. Ausprobieren mit `python bricktype.py`. Beobachtet dabei Folgendes:
 - `max(BrickType)` liefert das größte Enum-Mitglied – in diesem Fall `BrickType.TWICE`
 - `BrickType.BALL` könnt ihr statt der Zahl `2` verwenden – viel sprechender!
 - `BrickType.BALL` ist nicht einfach nur eine Zahl: Ihr könnt z. B. `.label()` darauf anwenden
 - Mit `BrickType(4)` könnt ihr einen ganz normalen Integer-Wert wieder in ein `BrickType`-Objekt umwandeln

2. **Was bringt uns dieses `(IntEnum)`?**

Entfernt in **Zeile 3** das `(IntEnum)` und führt das Programm nochmals aus:

- **Fehler in Zeile 14.** Die Funktion `max()` ist nicht mehr auf die Klasse anwendbar.

Kommentiert Zeile 14 aus und führt das Programm nochmals aus:

- Jetzt bekommt ihr einen neuen **Fehler in Zeile 16**. `BrickType.BALL` ist nur noch eine Zahl und die Methode `label` ist nicht mehr anwendbar.

Erkenntnis zum Effekt von `(IntEnum)` ist:

- Die Attribute wie `BrickType.BALL` sind nicht normale `int`-Werte, sondern Objekte vom Typ `BrickType`
- Diese Objekte haben Methoden wie `.label()`, die du selbst definierst
- Außerdem „erbst“ du gewisse Dinge, wie z. B. dass man `max(BrickType)` schreiben kann

Macht alle Änderungen wieder rückgängig: Mit den genauen Details kann man sich gerne an einem **ruhigeren Tag** beschäftigen – heute geht's ums **Verstehen durch Ausprobieren**, nicht um das letzte Python-Detail 😊

3. **Die `main()`-Funktion erweitern und `label()` verbessern**

Ändert die Funktion `main()` so ab:

```
def main():
    for i in range(0, max(BrickType)):
        bt = BrickType(i)
        print(f"i = {i}, bt = {bt}, bt.label = {bt.label()}")
```

Führt das Programm aus und vollzieht nach was passiert. Ändert nun die Methode `label()` so ab, dass sie – abhängig vom Wert von `self` – folgenden Text zurückgibt:

- `"Ball"` für `BrickType.BALL`
- `"Shield"` für `BrickType.SHIELD`
- `"Twice"` für `BrickType.TWICE`
- In allen anderen Fällen den leeren String `" "`

👉 Das ist eine kleine Übung in `if-elif-else`.

Schritt 4: Es lebe die Vielfalt (Teil 2 von 3)

Jetzt wird die Klasse `BrickType` **ganz sachte ins Spiel gebracht**. Zunächst geht es **nur um die Darstellung**: Bricks sollen sich **optisch unterscheiden**, indem ein kleiner **Label**-Text aufgedruckt wird.

Wichtig: Diese Bricks **verhalten sich noch nicht unterschiedlich** – das kommt später.



In `wall.py`

Importiert zuerst unsere neue Klasse mit `from bricktype import BrickType`.



Die `init`-Methode anpassen

Ändert dann die Methode `init` wie folgt:

```
12     def init():
13         for i in range(4, 8):
14             for j in range(0, Game.brick_cols):
15                 Wall.brick[i, j] = random.randint(1, max(BrickType))
```

Früher wurden dort alle Zellen stumpf mit `1` gefüllt – jetzt bekommt jede Zelle einen zufälligen `BrickType`-Wert (von `1` bis einschließlich `4`).



Die `draw`-Methode erweitern

Jetzt machen wir die neuen Brick-Typen sichtbar. Ergänzt in der `draw`-Methode folgenden Code ab **Zeile 45**:

```
33     def draw():
34         for i in range(0, Wall.brick.shape[0]):
35             for j in range(0, Wall.brick.shape[1]):
36                 if not Wall.brick[i, j]:
37                     continue
38                 w = Game.brick_width
39                 h = Game.brick_height
40                 x = j * w
41                 y = i * h
42                 r = (x + 1, y + 1, w - 1, h - 1)
43                 pygame.draw.rect(Game.screen, (223, 105, 165), r)
44
45                 label = BrickType.label(Wall.brick[i, j])
46                 font = pygame.font.SysFont(None, 24)
47                 text = font.render(label, True, (255, 255, 255))
48                 text_rect = text.get_rect(center=(x+w//2, y+h//2))
49                 Game.screen.blit(text, text_rect)
```

Hier hier jetzt der Label auf das Rechteck gezeichnet:

- **Zeile 45**: Ruft `BrickType.label(...)` auf und liefert einen normalen String zurück.
- **Zeile 46**: Erzeugt ein `Font`-Objekt (Schriftgröße 24, Standardschrift).
- **Zeile 47**: Wandelt den String in ein **Bitmap-Bild** um (Schriftfarbe: weiß, mit Antialiasing).
- **Zeile 48–49**: Zeichnet den Text **zentriert** in das Rechteck des Bricks.

Aufgabe

1. Testet, ob eure Anpassung funktioniert – also ob manche Bricks jetzt beschriftet sind. Ihr solltet z.B. "Ball", "Shield" oder "Twice" auf einzelnen Steinen lesen können.
2. Aktuell erscheinen **zu viele Spezial-Bricks** – das wirkt ein bisschen überladen. Besser wäre es, wenn mit einer gewissen Wahrscheinlichkeit (z. B. **20%**) ein Brick eine Spezialfunktion hat, und ansonsten einfach ein **normaler** Brick (NORMAL) gesetzt wird.

Vorschlag:

```
if random.randint(1, 100) <= 20:
    ty = random.randint(1, max(BrickType))
else:
    ty = 1
Wall.brick[i, j] = ty
```

Hinweis: In der Spezialauswahl sind dann BALL, SHIELD und TWICE gleich wahrscheinlich – das ist für unsere Zwecke völlig in Ordnung.

3. Es ist zwar nicht dramatisch – aber doch ein kleiner Stilbruch –, dass in jeder einzelnen draw()-Iteration ein neues Font-Objekt erzeugt wird. Auch wenn der Performanceverlust kaum messbar ist, gilt: Guter Code hat Anstand.

Besser wäre es, wenn das Font-Objekt einmalig zentral in der Game-Klasse erzeugt wird.



So geht's

In game.py, ergänzt in der Klasse Game ein neues Attribut:

```
class Game:
    # ... wie zuvor ...
    font = None
```

Dann passt ihr die init()-Methode wie folgt an:

```
def init():
    # ...
    pygame.init()
    # ...
    Game.font = pygame.font.SysFont(None, 24)
```

Wichtig: Das Font-Objekt darf erst nach pygame.init() erzeugt werden!

Anschließend könnt ihr in wall.py die Zeile

```
font = pygame.font.SysFont(None, 24)
```

ersetzen durch:

```
font = Game.font
```

4. Optionaler Feinschliff: Farbe nach Typ

In bricktype.py: Ergänzt die Klasse BrickType um eine Methode color, die – analog zur Methode label() – für jeden Brick-Typ eine passende Farbe zurückgibt.

In wall.py: Passt die draw()-Methode so an, dass jetzt die Farbe für den Brick vor dem Zeichnen aus der color()-Methode geholt wird:

```
c = BrickType.color(Wall.brick[i, j])
pygame.draw.rect(Game.screen, c, r)
```

Schritt 5: Es lebe die Vielfalt (Teil 3 von 3)

Unsere speziellen Bricks sollen jetzt nicht **nur anders aussehen**, sondern auch **einen Effekt haben**, wenn man sie trifft.

Wir gehen das Ganze **schrittweise** an – und fangen gleich mit dem „kompliziertesten“ Fall an: den **Bricks, die man zweimal treffen muss**.

Idee: Brick-Art bei Treffer verändern

Bricks vom Typ `TWICE` sollen sich **beim ersten Treffer** in `NORMAL` **verwandeln**, und beim zweiten Treffer dann **verschwinden**.

In `bricktype.py`

Erweitert die Klasse `BrickType` um eine Methode `effect()`, die zurückgibt, in was sich ein Brick verwandeln soll, wenn er getroffen wird:

```
def effect(self):
    if self == BrickType.TWICE:
        return BrickType.NORMAL
    else:
        return BrickType.NONE
```

In `wall.py`

In der Methode `collision()` wurde bisher ein getroffener Brick einfach gelöscht, z. B. so:

```
if Wall.brick[i2, j2]:
    Wall.brick[i2, j2] = 0
```

Ersetzt das jetzt durch:

```
if Wall.brick[i2, j2]:
    Wall.brick[i2, j2] = BrickType.effect(Wall.brick[i2, j2])
```

Damit entscheidet jeder Brick selbst, was mit ihm beim Treffer passiert.

Aufgabe

1. Teste, ob Bricks vom Typ `TWICE` jetzt erst beim zweiten Treffer verschwinden.

Pro-Tipp: Um das gezielt zu testen, kannst du in der `init()`-Methode von `Wall` **alle Bricks kurzfristig auf `TWICE` setzen**, z. B.:

```
Wall.brick[i, j] = 4 # ty
```

So sparst du dir das langwierige Suchen nach dem richtigen Brick – und siehst sofort, ob das Verhalten stimmt.

2. **Der einfache Fall? Denkste.**

Eigentlich wäre jetzt alles ganz leicht: Wenn ein Brick vom Typ `BALL` getroffen wird, dann soll **ein neuer Ball ins Spiel kommen**.

Klingt simpel: Einfach ein neues `BALL`-Objekt erzeugen, an `Game.balls` anhängen, fertig!
Aber denkste. Python macht den **eigentlich leichten Fall** leider kompliziert.

Experiment: Was geht schief?

Fügt in `bricktype.py` oben die Zeile ein:

```
from ball import Ball
```

Macht sonst keine weiteren Änderungen – und startet das Spiel. Es wird nicht funktionieren.

Schritt 6: Zyklische Abhängigkeiten

Was ist im letzten Schritt am Ende **schiefgegangen**? Ihr habt folgende **Import-Kette** erzeugt:

- `bricktype.py` importiert `ball.py`
- `ball.py` importiert `wall.py`
- `wall.py` importiert `bricktype.py`



Und da beißt sich die Katze in den Schwanz.

In einer „idealen“ objektorientierten Sprache kann jede Klasse **beliebig auf andere verweisen** – das ist ja gerade der Sinn von modularer Entwicklung. Aber in **Python** wird beim Importieren **sofort der ganze Code ausgeführt** – und weil Python nicht sauber zwischen **Interface** und **Implementierung** trennt, entstehen bei **zyklischen Abhängigkeiten** technische Probleme.



Notlösungen (nicht empfohlen)

Eine unsaubere Lösung wäre, (fast) **alle** `from ... import ...`-Statements durch einfache `import ...`-Statements zu ersetzen und später z. B. `modul.Klasse` zu verwenden. Das **umgeht** das Problem – ist aber nicht schön und bricht mit dem Stil.

Eine weitere (noch fragwürdigere) Möglichkeit wäre, das `from ball import Ball` direkt in die Methode zu schreiben, wo der Ball gebraucht wird – also z. B. in `BrickType.effect`.

Das wäre eine schnelle Lösung, aber ganz ehrlich: Das bricht mir mein Herz. Bei jedem Aufruf würde Python prüfen, ob das Modul geladen werden muss. Auch wenn das technisch gecached wird: Man muss Performance ja nicht mit Füßen treten.



Etwas saubere Lösung: Kontrolle der Imports

Wir unterbrechen den Import-Kreislauf, indem wir `BrickType` in `wall.py` nicht mehr direkt importieren, sondern zur Laufzeit an `Wall` übergeben.



Änderungen in `wall.py`

Entfernt oben die Zeile: `from bricktype import BrickType`

Ersetzt den Beginn der Klasse durch:

```
class Wall:
    brick = numpy.zeros((Game.brick_rows, Game.brick_cols))
    BrickType = None
```

Passt die Methode `init()` so an, dass `BrickType` übergeben und gespeichert wird:

```
def init(BrickType):
    Wall.BrickType = BrickType
    for i in range(4, 8):
        for j in range(0, Game.brick_cols):
            if random.randint(1, 100) <= 20:
                ty = random.randint(2, max(Wall.BrickType))
            else:
                ty = 1
            Wall.brick[i, j] = ty
```

Ändert den restlichen Code in `wall.py` so ab, dass immer `Wall.BrickType` statt `BrickType` verwendet wird.



Änderungen in `breakout.py`

Oben importiert ihr wie gewohnt `from bricktype import BrickType` und statt `Wall.init()` ruft ihr jetzt `Wall.init(BrickType)` auf.

Macht die Änderungen wie beschrieben und testet, ob alles weiterhin funktioniert. Wenn ja, habt ihr jetzt ein sauberes und zyklusfreies Design, und seid bereit für den nächsten Schritt

Schritt 7: Jetzt geht's wieder weiter mit Features

Nachdem wir das Problem mit den zyklischen Abhängigkeiten elegant gelöst haben, können wir in `bricktype.py` jetzt endlich alles importieren, was wir brauchen – (zumindest alles, außer aus `breakout.py` 😊).

Für den nächsten Schritt reicht:

```
from ball import Ball
from game import Game
```

`effect()`-Methode anpassen

Jetzt könnt ihr die Methode `effect()` in der Klasse `BrickType` wie geplant erweitern:

```
def effect(self):
    if self == BrickType.TWICE:
        return BrickType.NORMAL
    elif self == BrickType.BALL:
        Game.balls.append(Ball(100, 0, -3, 4))
        return BrickType.NONE
    else:
        return BrickType.NONE
```

Test

Mit dieser Änderung sollte beim Zerstören eines Bricks vom Typ `BALL` ein **neuer Ball** ins Spiel kommen.

Um das gezielt zu testen, könnt ihr – wie schon in früheren Aufgaben – in `wall.py` die Bricks vorübergehend so initialisieren:

```
Wall.brick[i, j] = 2 # ty
```

Dann seht ihr sofort, ob bei einem Treffer ein neuer Ball erzeugt wird.

Aufgabe

Beim Testen freut ihr euch vielleicht nur kurz darüber, dass ein neuer Ball ins Spiel kommt. Denn dummerweise entsteht er **oberhalb der Brick-Wall** – und sorgt dann für ein recht langweiliges Spielgeschehen, weil er einfach **Bricks von oben nach unten** abräumt, bis ein großes Loch durchschlagen ist.

Idee

Wir ändern das Spielverhalten so, dass ein neu erzeugter Ball zunächst inaktiv ist – d. h. er fliegt durch die Bricks hindurch, bis er vom Paddle getroffen wird. Erst dann wird er aktiv und kann Bricks zerstören.

Umsetzung in `ball.py`

1. Im Konstruktor (`__init__`) bekommt der Ball ein neues Attribut:
`self.active = False`
2. In der `move()`-Methode:
 - Ruft `Wall.collison(...)` nur auf, wenn `self.active == True`
 - Wenn der Ball das Paddle trifft, setzt `self.active = True`

Startet das Spiel und prüft, ob ein neuer Ball jetzt zuerst durch die Brick-Wall fällt, und erst nach dem Kontakt mit dem Paddle Bricks zerstören kann.

Schritt 8: Schutzschild

Jetzt bekommt das Paddle noch ein weiteres Feature: einen Schutzschild, der für eine gewisse Zeit Bombenschaden verhindert.

Vorbereitung in `paddle.py`

Zuerst wird die Klasse Paddle um ein neues Attribut `shield` erweitert, ähnlich wie `damage`. Dieses zählt herunter, solange der Schild aktiv ist:

```
class Paddle:
    # ...
    damage = 0
    shield = 0
```

Methode `hit()` anpassen

Der Schild schützt vor Schaden – daher muss `hit()` so angepasst werden:

```
def hit():
    if Paddle.damage == 0 and Paddle.shield == 0:
        Paddle.damage = 90
        return True
    return False
```

Neue Methode `set_shield()`

Mit dieser Methode wird der Schild aktiviert:

```
def set_shield():
    Paddle.shield = 300
```

Kosmetik in `draw()`

Wenn der Schild aktiv ist, wird das Paddle z. B. weiß gezeichnet:

```
def draw():
    if Paddle.damage > 0:
        Paddle.damage -= 1
        return
    if Paddle.shield > 0:
        Paddle.shield -= 1
        c = (255, 255, 255)
    else:
        c = Paddle.c
    pygame.draw.rect(Game.screen, c,
                     (Paddle.x, Paddle.y, Paddle.w, Paddle.h))
```

Aufgabe

1. Nehmt die Änderungen wie oben beschrieben in `paddle.py` vor.
2. Passt dann in `bricktype.py` die Methode `effect()` so an, dass Bricks vom Typ `SHIELD` unterstützt werden.

Das Vorgehen kennt ihr inzwischen:

- **Überlegt euch**, was ihr zusätzlich importieren müsst.
- **Überlegt euch**, wie ihr schnell testen könnt, ob das Feature funktioniert.

Wenn alles klappt, sollte das Paddle beim Zerstören eines `SHIELD`-Bricks für 10 Sekunden unverwundbar gegen Bomben sein – und dabei strahlend weiß leuchten ⚡

Schritt 9: Schutzschild

Wäre es nicht toll, wenn man sich auch **aktiv gegen Bomben wehren** könnte – also sie einfach **abschießt**, bevor sie einen treffen?



Oje.

Ja, das wäre toll... Aber **sollte man wirklich auf den letzten Drücker** noch ein neues Feature einbauen, anstatt wichtige Punkte auf der To-do-Liste abzuarbeiten?

Nein. **Sollte man nicht.**



Wie im echten Leben...

... machen wir's **trotzdem**. Und eigentlich ist es gar nicht so wild – es müsste ja **genau so funktionieren**, wie wenn man einen Spaceinvader vom Himmel holt.



Der bestehende Code in `breakout.py`

So werden aktuell Spaceinvader von Bällen oder Raketen getroffen:

```
for s in Game.ships:
    for item in Game.balls + Game.rockets:
        dist = math.hypot(item.x - s.x, item.y - s.y)
        if dist <= 15:
            s.hit = True
            item.hit = True
```

Durch die in den HM-Vorlesungen geschärfte Denkweise erkennen wir sofort: Wenn Bomben **genau so** wie Schiffe zerstört werden sollen, dann müssen wir einfach auch **über die Bomben iterieren**.

Also genügt es, die Schleife:

```
for s in Game.ships:
```

zu ersetzen durch:

```
for s in Game.ships + Game.bombs:
```

Aufgabe

Ist klar, oder? **Machen.** 😊

Schritt 10: Weil hier noch ein bisschen Platz ist

Wenn alle Spaceinvader abgeschossen sind, wird es langweilig – wir wollen ja Breakout **und** Spaceinvader spielen. Also bauen wir schnell noch das Feature ein, dass bei jedem getroffenen Brick ein neuer Spaceinvader entsteht.

Fügt in `bricktype.py` in der Methode `effect()` ganz am Anfang die Zeile ein:

```
Game.ships.append(Ship(0))
```

Das sollte eigentlich reichen.

Testet es einfach.

Falls ein Import-Dings fehlt, merkt ihr es schon an einer Fehlermeldung 😊

Schritt 11: Schaut mal auf die Uhr ...

... vielleicht sollten wir mal mit einem der wichtigen Features anfangen. Lasst uns also kurz innehalten und überlegen: **Was gehört bei einem Spiel wie Breakout + Spaceinvader unbedingt dazu?** Vielleicht ja, dass man nur eine begrenzte Anzahl an Leben hat – und man darum kämpft, ins nächste Level zu kommen, bevor alle Leben aufgebraucht sind. Also genau das, was ein Studium mit seiner begrenzt erlaubten Anzahl an Prüfungsversuchen so spannend macht.

Okay, wie fangen wir an? Wir müssen auf jeden Fall **Buch führen** über zwei wichtige Dinge:

- die Anzahl der **Leben**
- das aktuelle **Level**



In game.py

Fügt in der Klasse Game zwei neue Attribute hinzu:

```
class Game:
    # ... andere Attribute wie zuvor
    lifes = 0
    level = 0
```



Und dann: Anzeigen, was Sache ist

Damit man auch sieht, in welcher Lage man steckt, blenden wir `level` und `lifes` oben links im Fenster ein. Zum Glück habt ihr heute schon gesehen, wie man Text rendert – also könnt ihr in der `draw()`-Methode am Ende z. B. folgendes ergänzen:

```
def draw():
    # ... alles wie zuvor
    label = f"Level {Game.level} Lifes {Game.lifes}"
    text = Game.font.render(label, True, (255, 255, 255))
    text_rect = text.get_rect(topleft=(10, 10))
    Game.screen.blit(text, text_rect)
```



Test

Wenn ihr diese zwei Änderungen gemacht habt, solltet ihr oben links sehen, dass ihr **0 Leben** habt und im **Level 0** seid. Klingt erst mal ernüchternd – aber: **es ist ein Anfang**.



Und jetzt: Endlosschleife

Jetzt wollen wir das Spiel **robust weiterspielen lassen**, also einen einfachen Neustartmechanismus einbauen:

- Wenn man kein Leben mehr hat, startet das Spiel wieder von vorne. Zum Beispiel mit 3 Leben in Level 1
- Wenn kein Ball mehr im Spiel ist, verliert man ein Leben (wenn man noch eines hat), und es kommt ein neuer Ball ins Spiel.



Umsetzung in breakout.py

Startet mit einer leeren Ball-Liste: `Game.balls = []`. Innerhalb der Hauptschleife (z. B. direkt nach `while running:`) setzt ihr die Idee um mit:

```
if len(Game.balls) == 0:
    if Game.lifes > 1:
        Game.lifes -= 1
    else:
        Game.lifes = 3
        Game.level = 1
        Wall.init(BrickType)
        Game.balls.append(Ball(400, 0, -2, 6))
```

Aufgabe

1. Setzt die oben beschriebenen Änderungen um und testet das Spiel. Ihr solltet jetzt mit 3 Leben in Level 1 starten (das ist schon mal eine Verbesserung). Wenn alle Bälle verloren gegangen sind, verliert ihr ein Leben.
2. Prima, man kann jetzt Leben verlieren – aber noch keine gewinnen. Das mag im echten Leben so sein, aber in einem Spiel will man mehr. Zum Beispiel: dass man in den nächsten Level aufsteigt, wenn alle Bricks abgeräumt sind – und dann vor einer noch größeren Herausforderung steht, etwa einer zusätzlichen Brick-Reihe.

Ändert dazu in `wall.py` die `init`-Methode so, dass die Anzahl der Brick-Reihen dem aktuellen Level entspricht:

```
def init(BrickType):
    Wall.BrickType = BrickType
    for i in range(4, 4 + Game.level):
        for j in range(0, Game.brick_cols):
            if random.randint(1, 100) <= 20:
                ty = random.randint(2, max(Wall.BrickType))
            else:
                ty = 1
            Wall.brick[i, j] = ty
```

In `breakout.py` entfernt ihr zunächst die alte Initialisierung mit `Wall.init(BrickType)`. Stattdessen könnt ihr jetzt in der Hauptschleife regelmäßig abfragen, wie viele Bricks noch im Spiel sind. Wenn keine mehr da sind, steigt man einen Level auf:

```
if Wall.brick.sum() == 0:
    Game.level += 1
    Wall.init(BrickType)
```

Die Methode `sum` stammt aus `numpy`, das – wie ihr wisst – ein Wrapper für eine sehr effiziente C/C++-Bibliothek für numerische lineare Algebra ist. Die Summe aller Matrixelemente zu berechnen ist extrem effizient. Es lohnt sich also nicht, auf Python-Seite extra mitzuzählen, wie viele Bricks noch vorhanden sind. Der Code ist kurz, klar und performant.

Macht diese Änderungen und testet das Spiel. Ihr beginnt jetzt mit einer Brick-Reihe. Wenn ihr es schafft, diese komplett abzuräumen, steigt ihr einen Level auf und steht einer neuen Wand mit einer zusätzlichen Reihe gegenüber. Wenn ihr alle Leben verloren habt, beginnt ihr wieder mit 3 Leben in Level 1.

Schritt 12: Weil hier noch Platz ist, gibt's auf die Ohren

Die PCs in den Laborräumen haben leider keine Lautsprecher. Aber: Es gibt Kopfhörer! Und wer Lust hat, dem Spiel etwas mehr Wumms zu verpassen, kann jetzt noch Soundeffekte einbauen.

Mit `pygame` geht das ganz einfach. Aus einer Audio-Datei wird ein Soundobjekt erzeugt – das ihr mit `.play()` abspielen könnt, wann immer euch danach ist.

Zum Beispiel in `game.py` in der `init`-Methode:

```
Game.exploding = pygame.mixer.Sound("data/exploding.wav")
Game.bounce = pygame.mixer.Sound("data/bounce.mp3")
Game.launch = pygame.mixer.Sound("data/launch.wav")
```

Wenn dann z. B. eine Rakete abgefeuert wird, lässt sich der passende Clip so abspielen:

```
Game.launch.play()
```

Mehr Platz für Erklärungen ist hier nicht – aber das macht nichts. Denn an dieser Stelle ist Experimentierfreude gefragt. Überlegt euch wo man weitere Soundeffekte abspielen kann.

Schritt 13: Ach ja, das Repository ...

Falls es jemand bis hierher geschafft hat: Jetzt wäre es langsam an der Zeit, die Änderungen ins Repository zu übernehmen – falls ihr nicht sowieso schon selbst auf die Idee gekommen seid 😊

```
git add *.py
git commit -a -m "wird Zeit"
git push
```

Schritt 14: Und dann noch die Sache mit dem Controller

Im Labor haben wir euch (wenn wir es nicht vergessen haben) einen Game-Controller mit einem BNO055-Sensor zur Verfügung gestellt. Damit könnt ihr das Paddle ganz lässig mit dem Handgelenk steuern.

Überraschend – oder vielleicht auch nicht – sollte sein, dass ihr euren Python-Code zum Auslesen der seriellen Schnittstelle gar nicht anpassen musstet. Warum? Weil es völlig egal ist, ob der Mikrocontroller ein Potentiometer oder einen anderen Sensor ausliest – solange er die Daten im gleichen Protokoll über die serielle Schnittstelle schickt, klappt alles wie gehabt.

Wer den passenden Mikrocontroller-Code für den BNO055 haben möchte:

```
git clone git@github.com:michael-lehn/asp-ulm-bno055.git
```

Das Makefile funktioniert so, wie ihr es inzwischen kennt:

- `make` kompiliert den Code
- `make upload` kompiliert und flasht
- `make monitor` startet den seriellen Monitor

Schritt 15: Und hier wäre noch Platz ...

... für eine lange To-do-Liste mit Refactoring, Clean Code, Menü, Highscore und Optionen. Oder für ein paar abschließende Worte über den Weg, den wir gegangen sind.

Aber ganz ehrlich: Ich mag weder To-do-Listen noch Abschiede 😊